Nome: João Pedro Ramos Moraes
Roteiro 3: Enum, Struct e Ponteiros

1. Crie uma enum para dias da semana e uma struct que armazene o nome de uma tarefa e o dia da semana em que deve ser feita. Grave e leia do arquivo texto.

```c
#include <stdio.h>

enum dia { DOM, SEG, TER, QUA, QUI, SEX, SAB };

struct tar {
    char n[50];
    enum dia d;
};

int main() {
    struct tar t1 = {"Estudar", SEG};
    struct tar t2;
    int x;
    FILE *f;

    f = fopen("f.txt", "w");
    fprintf(f, "%s %d\n", t1.n, t1.d);
    fclose(f);

    f = fopen("f.txt", "r");
    fscanf(f, "%s %d", t2.n, &x);
    t2.d = x;
    fclose(f);

    printf("%s %d\n", t2.n, t2.d);

    return 0;
}
```

2. Defina uma enum com níveis de acesso (ADMIN, USER, GUEST) e uma struct Usuario contendo nome e nível. Implemente função que recebe ponteiro e altera o nível.

```c
#include <stdio.h>

enum Nivel { ADMIN, USER, GUEST };

struct Usuario {
    char nome[50];
    enum Nivel nivel;
};

struct Usuario alterarNivel(struct Usuario user,
enum Nivel novoNivel) {
    user.nivel = novoNivel;
    return user;
}

int main() {
    struct Usuario user1 = {"Ana", GUEST};

    user1 = alterarNivel(user1, ADMIN);

    printf("%s %d\n", user1.nome, user1.nivel);

    return 0;
}
```

3. Crie uma struct Veiculo com marca, modelo e tipo (enum Tipo {CARRO, MOTO, CAMINHAO}). Leia 5 veículos e salve em arquivo binário.

```c
#include <stdio.h>

enum Tipo { CARRO, MOTO, CAMINHAO };

struct Veiculo {
    char marca[50];
    char modelo[50];
    enum Tipo tipo;
};

int main() {
    struct Veiculo veiculos[5];
    int i;
    int tipoTemp;
    FILE *arquivo;

    for(i = 0; i < 5; i++) {
        scanf("%s", veiculos[i].marca);
        scanf("%s", veiculos[i].modelo);
        scanf("%d", &tipoTemp);
        veiculos[i].tipo = tipoTemp;
    }

    arquivo = fopen("veiculos.bin", "wb");
    fwrite(veiculos, sizeof(struct Veiculo), 5,
arquivo);
    fclose(arquivo);

    return 0;
}
```

4. Implemente função que recebe vetor de struct Aluno e imprime apenas os alunos cujo status (enum Status {APROVADO, REPROVADO}) é APROVADO.

```c
#include <stdio.h>

enum Status { APROVADO, REPROVADO };

struct Aluno {
    char nome[50];
    enum Status status;
};

void imprimirAprovados(struct Aluno alunos[], int
tamanho) {
    int i;
    for(i = 0; i < tamanho; i++) {
        if(alunos[i].status == APROVADO) {
            printf("%s\n", alunos[i].nome);
        }
    }
}

int main() {
    struct Aluno alunos[3] = {
        {"Joao", APROVADO},
        {"Maria", REPROVADO},
        {"Pedro", APROVADO}
    };

    imprimirAprovados(alunos, 3);

    return 0;
}
```

5. Crie função que recebe ponteiro para struct Produto e aplica desconto de acordo com categoria (enum Categoria {ELETRONICO, ROUPA, ALIMENTO}).

```c
#include <stdio.h>

enum Categoria { ELETRONICO, ROUPA, ALIMENTO };

struct Produto {
    char nome[50];
    float preco;
    enum Categoria categoria;
};

struct Produto aplicarDesconto(struct Produto
produto) {
    if(produto.categoria == ELETRONICO) {
        produto.preco = produto.preco * 0.90;
    } else if(produto.categoria == ROUPA) {
        produto.preco = produto.preco * 0.80;
    } else if(produto.categoria == ALIMENTO) {
        produto.preco = produto.preco * 0.95;
    }
    return produto;
}

int main() {
    struct Produto produto1 = {"Celular", 1000.0,
ELETRONICO};

    produto1 = aplicarDesconto(produto1);

    printf("%s %.2f\n", produto1.nome,
produto1.preco);

    return 0;
}
```

6. Desenvolva programa que leia um vetor de struct Conta e utilize enum Operacao {SAQUE, DEPOSITO} para registrar operações em arquivo.

```c
#include <stdio.h>

enum Operacao { SAQUE, DEPOSITO };

struct Conta {
    int numero;
    enum Operacao operacao;
    float valor;
};

int main() {
    struct Conta contas[3];
    int i;
    int operacaoTemp;
    FILE *arquivo;

    for(i = 0; i < 3; i++) {
        scanf("%d", &contas[i].numero);
        scanf("%d", &operacaoTemp);
        contas[i].operacao = operacaoTemp;
        scanf("%f", &contas[i].valor);
    }

    arquivo = fopen("operacoes.txt", "w");

    for(i = 0; i < 3; i++) {
        fprintf(arquivo, "%d %d %.2f\n", contas[i].numero,
contas[i].operacao, contas[i].valor);
    }

    fclose(arquivo);

    return 0;
}
```

7. Crie função que receba ponteiro para vetor de struct Pessoa e retorne a quantidade de pessoas de um determinado gênero (enum Genero).

```c
#include <stdio.h>

enum Genero { MASCULINO, FEMININO, OUTRO };

struct Pessoa {
    char nome[50];
    enum Genero genero;
};

int contarGenero(struct Pessoa pessoas[], int tamanho, enum Genero alvo)
{
    int contador = 0;
    int i;

    for(i = 0; i < tamanho; i++) {
        if(pessoas[i].genero == alvo) {
            contador++;
        }
    }

    return contador;
}

int main() {
    struct Pessoa pessoas[4] = {
        {"Joao", MASCULINO},
        {"Maria", FEMININO},
        {"Carlos", MASCULINO},
        {"Ana", FEMININO}
    };

    int quantidade = contarGenero(pessoas, 4, MASCULINO);

    printf("%d\n", quantidade);

    return 0;
}
```

8. Implemente struct Funcionario com cargo definido por enum Cargo. Grave 10 funcionários em arquivo binário e depois leia-os listando apenas GERENTES.

9. Crie vetor de struct Sensor com status (enum Estado {LIGADO, DESLIGADO}). Escreva função que altere o estado via ponteiro e salve as mudanças no arquivo.

10. Implemente menu com enum Opcoes {CADASTRAR, LISTAR, BUSCAR, SAIR} controlando as funções de um cadastro de clientes (usando structs e arquivo).

11. Crie vetor dinâmico de struct Animal com tipo (enum TipoAnimal). Use ponteiros para inserir e remover animais dinamicamente.
12. Desenvolva função que leia arquivo binário com struct Pedido e exiba os pedidos cujo status (enum StatusPedido {PENDENTE, ENVIADO, ENTREGUE}) seja ENVIADO.
13. Implemente função que receba ponteiro para vetor de struct Carro e calcule média de preço por tipo (enum CategoriaCarro).
14. Crie struct Jogo com gênero (enum GeneroJogo) e use ponteiros para ordenar vetor por nome antes de salvar em arquivo texto.
15. Desenvolva sistema simples de biblioteca com struct Livro, enum CategoriaLivro e funções para cadastrar, listar, buscar e salvar em arquivo binário.